Clase 18 · Redes neuronales

Reconocimiento de Patrones · Ing. Rubén Omar Azuara Domínguez · lunes 26 oct 2026 (sesión 21 del curso) · Apuntes y resumen de estudio

PARTE 1 · Apuntes completos

1. Por qué pasar de Boosting a redes neuronales

Gradient Boosting (Clase 17) es un ensamble secuencial: cada modelo corrige los residuos del anterior con descenso de gradiente en el espacio de funciones. Su límite es que no modela dos tipos de dependencia:

- **Secuencial o temporal:** audio, series de tiempo.
- **Espacial:** la vecindad entre píxeles de una imagen.

Por eso, con imágenes o señales crudas hay que extraer características a mano antes de usar Boosting. Las redes neuronales profundas aprenden esas representaciones directo de los datos crudos.

2. Historia y estructura de una neurona

Una red neuronal es un modelo computacional inspirado en la organización de las neuronas del cerebro. No es una tecnología nueva:

Año	Hecho
1957–1958	Frank Rosenblatt diseña (1957) y publica (1958) el **perceptrón**.
1969	Marvin Minsky y Seymour Papert demuestran que una sola neurona no aprende problemas no lineales como **XOR**; el área se frena por años.
Hoy	El resurgimiento se debe al cómputo en paralelo y a las **GPU**: las matemáticas son de hace décadas, el hardware las hizo viables.

Neurona biológica	Neurona artificial
Dendritas: reciben señales	Entradas X
Núcleo: procesa	Suma ponderada con pesos W y sesgo b
Axón y terminal: transmite	Función de activación φ y salida Y

3. El perceptrón simple

Definición

El perceptrón combina linealmente entradas y pesos, suma un sesgo y aplica una función de activación: $\mathbf{Y = \varphi(\Sigma\, w_i x_i + b) = \varphi(W^T X + b)}$.

Ejemplo resuelto en clase — compuerta AND. Pesos $w_1 = w_2 = 1$, sesgo $b = -1.5$, activación escalón ($\varphi(z) = 1$ si $z \geq 0$; 0 si $z < 0$).

x_1	x_2	$z = x_1 + x_2 - 1.5$	Salida
0	0	−1.5	0
1	0	−0.5	0
0	1	−0.5	0
1	1	+0.5	1

Limitación: solo traza una frontera recta, así que aprende AND y OR pero **no XOR**.

4. Perceptrón multicapa (MLP) y funciones de activación

El MLP tiene capa de entrada (características crudas), una o más **capas ocultas** (representaciones cada vez más abstractas) y capa de salida (la decisión). Así resuelve XOR y fronteras arbitrariamente complejas.

> **Regla clave**
>
> Las capas ocultas deben tener activación **no lineal** (sigmoide, tanh, ReLU). Con activaciones lineales, la composición de capas se reduce a una sola combinación lineal y la red profunda colapsa a un modelo lineal.

Tarea	Activación de salida	Función de pérdida
Regresión	Lineal	Error cuadrático medio (MSE)
Clasificación binaria	Sigmoide (entre 0 y 1)	Entropía cruzada binaria
Multiclase (K clases)	K neuronas con softmax (suman 1)	Entropía cruzada multiclase

Error común: usar sigmoide en una regresión no acotada, o activaciones lineales en las ocultas.

5. Entrenamiento: pérdida, SGD, minilotes y épocas

Entrenar es ajustar pesos y sesgos para minimizar una pérdida sobre una superficie **no convexa**, con mínimos locales y **puntos silla** (derivada cero sin ser mínimo). El ciclo tiene cuatro pasos:

1. **Forward pass:** la entrada pasa por pesos, sesgos y activaciones hasta la predicción $\hat{y}$.
2. **Pérdida:** se compara $\hat{y}$ con la etiqueta real y.
3. **Backward pass (retropropagación):** derivadas de la pérdida respecto a cada peso con la regla de la cadena, de la salida a la entrada.
4. **Actualización: $W_{nuevo} = W_{viejo} - \alpha \cdot \nabla L$**, con tasa de aprendizaje α.

Procesar todo el conjunto de golpe es inviable, así que se usa **SGD con minilotes** de tamaño B. Una **época** es una pasada completa por todos los minilotes; en cada época los datos se **revuelven** para que la red no memorice el orden. Un minilote demasiado grande desborda la memoria de la GPU.

6. Retropropagación paso a paso

Red de ejemplo: entrada X_1, peso $W_{11}^{(1)}$, sesgo $b_1^{(1)}$, activación A_1, peso de salida $W_{11}^{(2)}$ y pérdida $L = \frac{1}{2}(\hat{y} - y)^2$.

- Respecto a la salida: $\partial L/\partial \hat{y} = (\hat{y} - y)$.
- Última capa: $\partial L/\partial W_{11}^{(2)} = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial W_{11}^{(2)}$.
- Capa más profunda: $\partial L/\partial W_{11}^{(1)} = \partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial A_1 \cdot \partial A_1/\partial z_1 \cdot \partial z_1/\partial W_{11}^{(1)}$.

La clave de eficiencia: los términos ya calculados para la capa posterior ($\partial L/\partial \hat{y} \cdot \partial \hat{y}/\partial A_1$) **se reutilizan** para las anteriores en vez de recalcularse. Eso hace viable entrenar redes grandes.

7. Aproximación universal: ancho contra profundidad

> **Teorema de aproximación universal**
>
> Un MLP con **una sola capa oculta** y suficientes neuronas puede aproximar cualquier función continua.

El teorema asegura que la red existe, no que sea fácil de entrenar ni de tamaño razonable. **Ancho** = neuronas por capa; **profundidad** = número de capas. En la práctica una capa infinitamente ancha es inviable; apilar capas (22, 50, 100 o más) reduce los parámetros necesarios y aprende jerarquías de características. Es el paso del aprendizaje máquina tradicional al aprendizaje profundo.

8. Desvanecimiento y explosión del gradiente

Inestabilidad numérica de la retropropagación en redes muy profundas, porque el gradiente es un producto de muchas derivadas intermedias.

- **Desvanecimiento:** activaciones saturadas (sigmoide, tanh) tienen derivada casi cero en los extremos; el producto tiende a cero y las primeras capas dejan de aprender.
- **Explosión:** derivadas o pesos grandes hacen crecer el producto exponencialmente y desestabilizan las actualizaciones.

Solución	Qué hace
Activaciones no saturadas	ReLU, LeakyReLU, SiLU, Softplus
Conexiones residuales (ResNet)	Suman la entrada a la salida del bloque, f(x) + x; el gradiente fluye por el atajo
Batch Normalization	Normaliza las activaciones por lote
Gradient clipping	Recorta gradientes demasiado grandes
Inicialización He o Xavier	Pesos iniciales con la escala adecuada

Estas innovaciones permitieron entrenar redes de cientos de capas (ResNet, Transformers).

9. Sobreajuste, subajuste y regularización

Situación	Síntoma en las curvas
Sobreajuste	La pérdida de entrenamiento sigue bajando, la de validación empieza a subir

Subajuste	Ambas pérdidas se quedan altas y estáticas

Técnicas de regularización:

- **Paro temprano:** detener en el mínimo de la pérdida de validación.
- **L1 y L2 (weight decay):** penalizar el costo para que los pesos no crezcan sin control.
- **Dropout:** apagar al azar una proporción de neuronas en cada iteración (actúa sobre la arquitectura).
- **Aumento de datos:** rotaciones, volteos, recortes, cambios de color o escala (actúa sobre las muestras).
- **Suavizado de etiquetas:** no exigir probabilidades de exactamente 1.0.

10. Avisos de la clase

- **Práctica 4 (KNN y regresión logística):** según NotebookLM se sube a Classroom el viernes 30 oct y se expone el viernes siguiente (15–20 min por equipo, código y resultados, sin teoría). Sin contrastar con Classroom.
- **Proyecto final:** último viernes del semestre (27 nov 2026 según el audio; el sitio dice tentativo 30 nov). Opciones: emociones en imágenes (FER2013 o RAF-DB), emociones en audio (RAVDESS, 8 clases) o comandos de voz (Speech Commands, 35 clases), probado con fotos o audios del equipo.
- Próxima sesión: más redes neuronales y luego procesamiento de lenguaje natural con Transformers.

PARTE 2 · Resumen para estudiar

Versión corta de lo que más probablemente se pregunta. Léela primero y vuelve a la Parte 1 si algo no queda claro.

Motivación. Boosting no capta dependencias temporales (audio) ni espaciales (píxeles); las redes aprenden las representaciones solas.

Historia. Rosenblatt: perceptrón (1957/1958). Minsky y Papert (1969): una neurona no resuelve XOR. Hoy: GPU y cómputo paralelo.

> **Perceptrón (examen)**
>
> $Y = \varphi(W^T X + b)$. AND con $w = (1, 1)$, $b = -1.5$ y escalón: solo $(1, 1)$ da $z = +0.5 \rightarrow 1$. Separa linealmente: AND y OR sí, XOR no.

MLP. Capas ocultas con activación **no lineal**; si fueran lineales, la red colapsa a un modelo lineal.

Tarea	Salida + pérdida
Regresión	Lineal + MSE
Binaria	Sigmoide + entropía cruzada binaria
Multiclase	Softmax + entropía cruzada

Entrenamiento. Forward → pérdida → backward (regla de la cadena, reutilizando términos) → $W \leftarrow W - \alpha \cdot \nabla L$. SGD con minilotes de tamaño B; época = pasada completa; revolver en cada época.

Aproximación universal. Una capa oculta basta en teoría; en la práctica la profundidad es más eficiente.

Problema	Causa	Remedios
Desvanecimiento	Derivadas ≈ 0 (sigmoide, tanh)	ReLU y variantes, residuales f(x)+x, BatchNorm, He/Xavier
Explosión	Derivadas/pesos grandes	Gradient clipping, inicialización, BatchNorm
Sobreajuste	Validación sube mientras entrenamiento baja	Paro temprano, L1/L2, Dropout, aumento de datos, suavizado de etiquetas
Subajuste	Ambas pérdidas altas	Modelo más capaz, más entrenamiento